

Nakijk 01_03 — V3

NAKIJK V3 · Hoofdstuk 01 · Twee routeparameters in één route · bij `Oefening_01_03.md` · derde versie van je code, 4 oktober 2026

Opmerking

Dit is een nakijk van jouw code, geen oplossing. Er staat geen verbeterde code in. Per fout lees je waar ze zit, wat je API nu antwoordt, waarom, en in welke richting je moet zoeken.

Dit is **V3**: de nakijk van je derde versie. De eerste twee nakijken horen bij V1 en V2.

In één oogopslag

Je code is opnieuw getest met alle tien de voorbeelden uit de opgave, met `/api` ervoor. **Acht** geven letter voor letter het gevraagde antwoord. De twee die mislukken, staan allebei in punt 3, de voeding.

#	Fout uit V1 en V2	Status in V3
1	namen in de socketroute ≠ namen van de parameters	opgelost
2	namen in de voedingsroute ≠ namen van de parameters	terug open
3	slotenroute begon met <code>/</code> , en de namen verschilden	opgelost
4	montage had de verkeerde socket	opgelost
5	sloten en modules in de omgekeerde volgorde doorgegeven	opgelost
6	de naam van de build stond vooraan in het antwoord	opgelost

Opkuispunt uit V2	Status in V3
een <code>\$</code> voor een tekst zonder <code>{...}</code>	opgelost
een naam die zegt wat het voedingsendpoint teruggeeft	opgelost
dezelfde vraag voor de hulpmethode erachter	nog open
<code>{wat}</code> en <code>{gebruik}</code> hernoemen naar duidelijkere namen	half gedaan, zie fout 2
lege regel tussen attribuut en methode	nog open

A. Wat nog een verkeerd antwoord geeft

Fout 2 — de voedingsroute en de parameters zijn opnieuw uit elkaar gegroeid

Waar. Het attribuut boven `VoedingAdvies` en de kop van die methode.

Verzoek	Je API antwoordt nu	De opgave vraagt
GET <code>/api/compatibiliteit/voeding/550/620</code>	De voeding volstaat, maar de marge is krap.	De voeding is te zwak voor deze build.
GET <code>/api/compatibiliteit/voeding/650/620</code>	De voeding volstaat, maar de marge is krap.	De voeding volstaat, maar de marge is krap. (toevallig juist)

Verzoek	Je API antwoordt nu	De opgave vraagt
GET /api/compatibiliteit/voeding/850/620	De voeding volstaat, maar de marge is krap.	De voeding is ruim voldoende.

Waarom. Je hebt de parameters van `VoedingAdvies` hernoemd naar `wattage` en `verbruik`. Dat was de opkuis uit V2. Maar tussen de accolades in de route staan nog `{wat}` en `{gebruik}`. Dezelfde fout als in V1, alleen nu langs de andere kant.

ASP.NET Core koppelt een stuk van de URL alleen aan een parameter met **exact dezelfde naam**. Vindt het in de route geen `wattage`, dan zoekt het in de querystring, en daar staat niets. Een `int` die nergens gevonden wordt, krijgt stil de waarde 0, zonder foutmelding. Je endpoint rekent dus altijd met 0 en 0, en dat verschil valt in je `switch` in de tak van de krappe marge. Daarom krijg je bij elk voedingsverzoek hetzelfde antwoord, wat je ook in de URL zet.

Waarom punt 5 wel werkt. In `InformatieBuild` roep je `VoedingAdvies` gewoon op als methode. Daar speelt de route geen rol: de argumenten gaan op volgorde mee. Daardoor kan het voedingsendpoint fout zijn terwijl de build juist rekent.

Richting. Een naam van een routeparameter bestaat altijd op twee plaatsen. Leg het attribuut en de kop van de methode naast elkaar en lees ze naam per naam.

Zo vang je dit voortaan sneller

- Hernoem je een parameter die uit de route komt, pas dan in dezelfde beweging het attribuut aan. Doe daarna meteen een van de voorbeelden van dat endpoint opnieuw.
- Krijg je in een reeks verzoeken telkens exact hetzelfde antwoord, wat je ook invult? Dan komt je invoer waarschijnlijk niet aan.
- Open het endpoint in Scalar en vergelijk de invoervelden die het toont met de twee getallen die je in de URL zet. Kijk ook waar elk veld staat.

B. Werkt, maar wijkt af van de opgave

Ongewijzigd sinds V2.

Wat	Waarom het toch aandacht vraagt
Je route begint met <code>api/</code> .	De voorbeelden in de opgave staan zonder <code>api</code> . Verdedigbaar, de cursus raadt <code>api/</code> zelfs aan. Zet het dan wel voor elk voorbeeld als je test.
Bij één vrij slot zegt je API <i>slot</i> in plaats van <i>sloten</i> .	Beter Nederlands, maar de opgave vraagt letterlijk <i>Er blijven nog {aantal} sloten vrij</i> . Bij een examen houd je je aan de opgave.

C. Opkuis, zonder invloed op het antwoord

- `BerekenWattage` berekent geen wattage, maar geeft een voedingsadvies terug. Het endpoint heeft nu een naam die dat zegt, de hulpmethode erachter nog niet.
- De constante `Marge` staat tussen twee endpoints in. Constanten staan meestal bovenaan de klasse, zodat je meteen ziet met welke vaste waarden de klasse werkt.
- Tussen elk attribuut en zijn methode staat nog een lege regel. Dat werkt, maar meestal staat het attribuut er rechtstreeks boven, zodat je ziet dat ze bij elkaar horen.

D. Wat beter is dan in V2

- **Fout 5 is opgelost.** Sloten en modules gaan nu in de juiste volgorde mee, en beide voorbeelden uit punt 5 geven letter voor letter het gevraagde antwoord.
- Punt 2, punt 4 en punt 5 zijn volledig juist: zeven van de zeven voorbeelden.
- De overbodige \$ voor *De processor past op dit moederbord.* is weg.
- VoedingAdvies is een goede naam voor een endpoint: hij zegt wat je terugkrijgt.

E. Zelf nakijken

Gebruik enkel de voorbeelden uit `Oefening_01_03.md`, met `/api` ervoor.

Na het oplossen van	Doe deze verzoeken	Je moet krijgen
fout 2	de drie voorbeelden uit punt 3	drie verschillende antwoorden, elk letter voor letter gelijk aan de opgave
daarna	alle tien de voorbeelden: twee uit punt 2, drie uit punt 3, drie uit punt 4 en twee uit punt 5	tien keer exact het gevraagde antwoord

Klopt alles, leg je endpoints dan ook nog naast de nakijkpunten in `Oplossing_01_03.md`.